

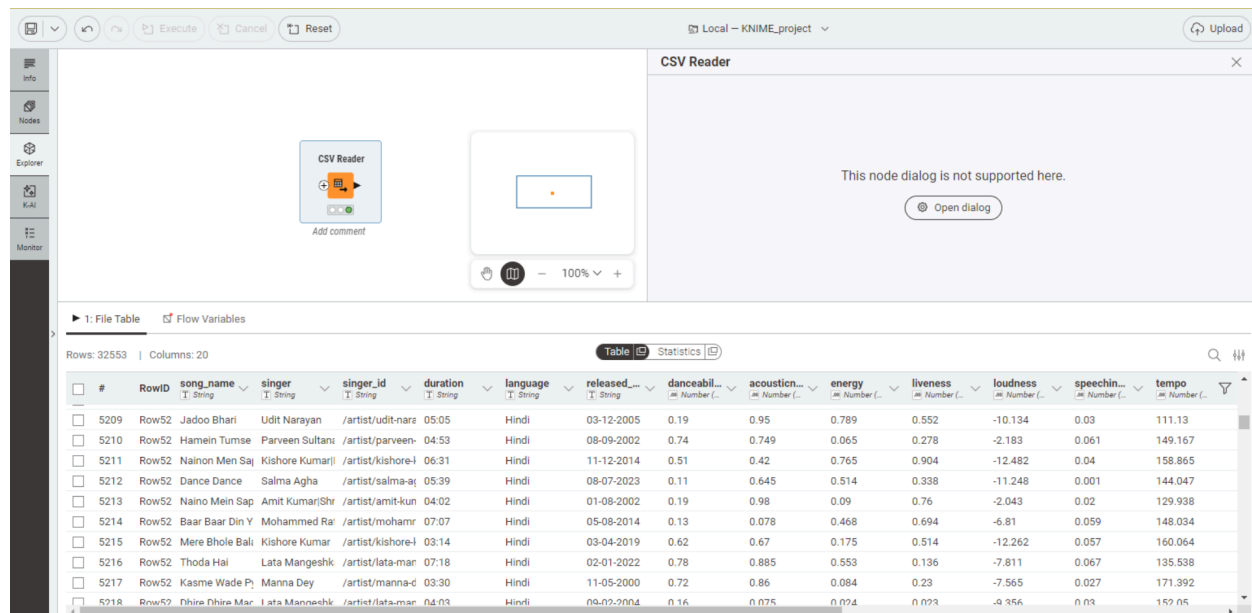
KNIME-Playlist-automation

Nayantara Ravi Samtani

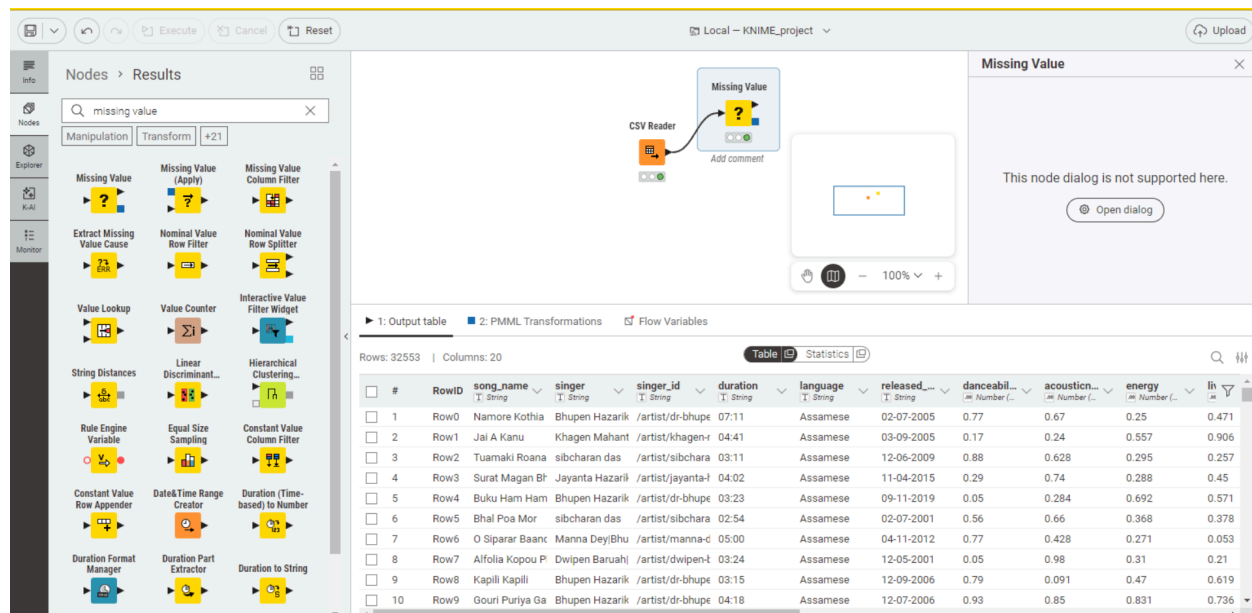
Lakehead University

BUSI-4016: Data Mining & Predictive Analytics

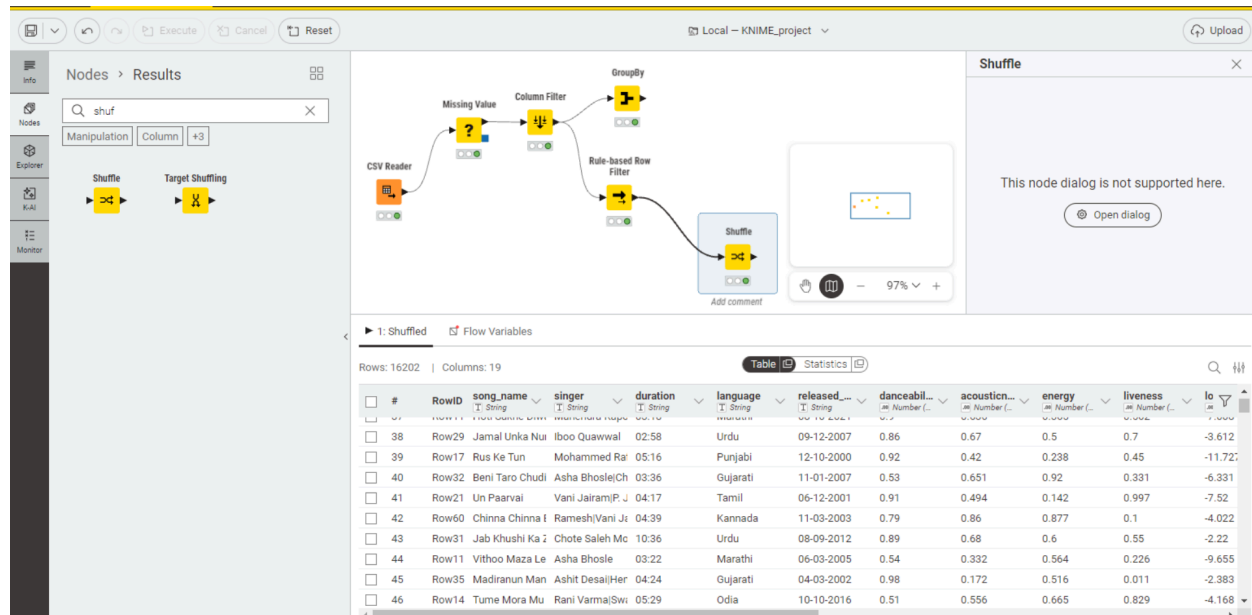
14-11-2025



To begin the workflow, I imported the dataset song.csv into KNIME using the **CSV Reader** node. This node loads the raw data into the KNIME workflow environment so that further cleaning and transformation steps can be performed.



After loading the dataset, I used the **Missing Value** node to clean the data. This step ensures that any rows containing null or missing values are handled to avoid errors during filtering and playlist creation. The removal of missing data improves the quality and reliability of analysis.



I created two separate branches by applying the **Rule-based Row Filter** node twice. For the

High Danceability branch, I used the rule: $\$danceability\$ \geq 0.8 \Rightarrow \text{TRUE}$

For the **Medium Danceability** branch, I used the rule: $\$danceability\$ \geq 0.6 \text{ AND}$

$\$danceability\$ < 0.8 \Rightarrow \text{TRUE}$

After creating the two branches, I applied the **Shuffle** node to each. This step randomizes the song order to avoid predictable sequencing and create natural playlist variation for the event.

Nodes > Results

Other Data Types | Manipulation | +24

Workflow: Rule-based Row Filter → Shuffle → Moving Aggregator

Results: 1: Moving average values

#	RowID	dance_level
1	Row0	high
2	Row1	high
3	Row2	high
4	Row3	high
5	Row4	medium
6	Row5	high
7	Row6	high
8	Row7	high
9	Row8	high
10	Row9	medium

Nodes > Results

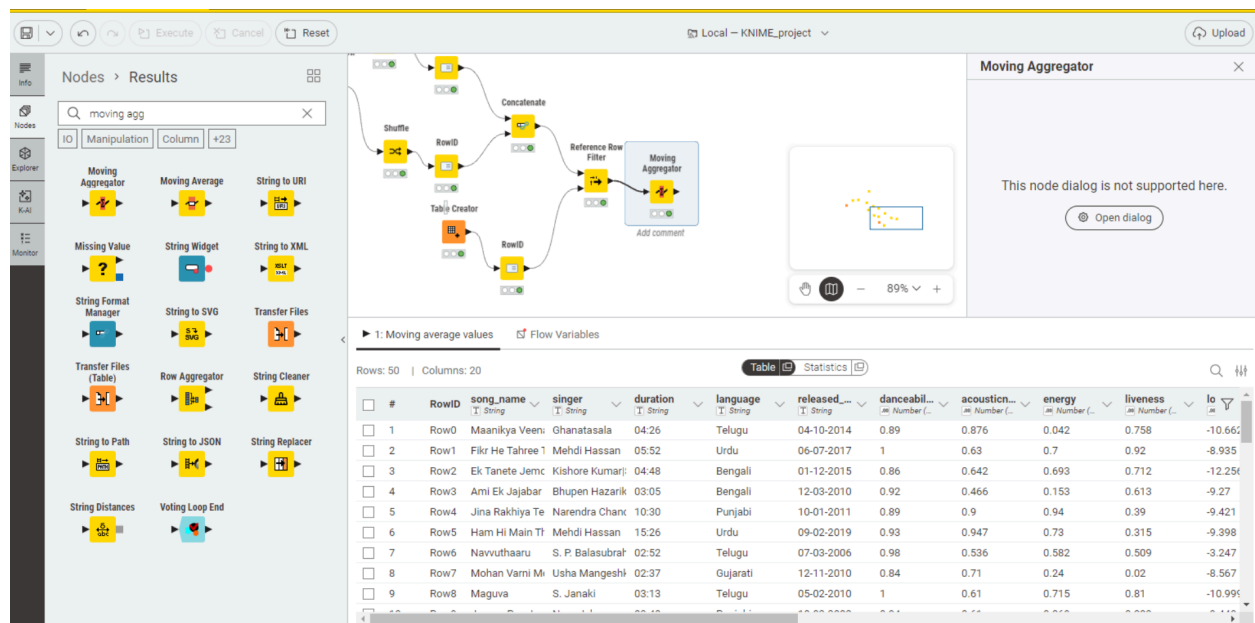
Manipulation | Column | Analytics | +26

Workflow: Rule-based Row Filter → Shuffle → Table Creator → RowID

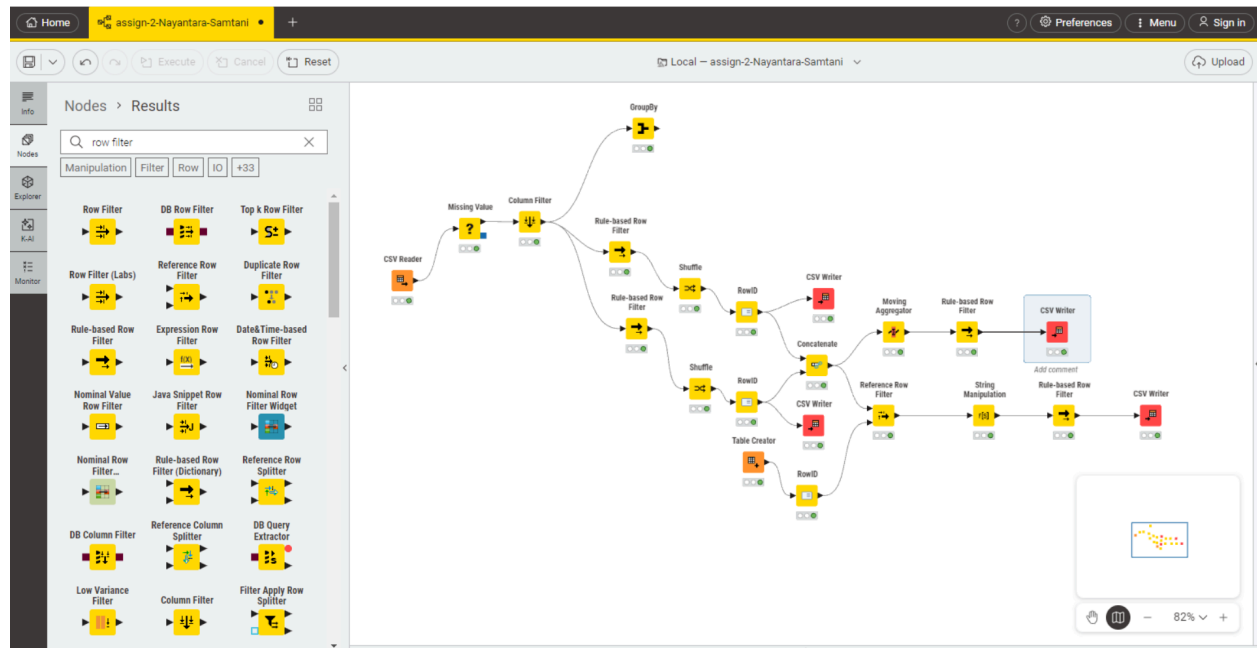
Results: 1: Output Table

#	RowID	song_name	singer	duration	energy	liveness	loudness
1	Row0	Maanikya Veen	Ghanatasala	04:26	0.042	0.758	-10.66
2	Row1	Fikr He Tahree 1	Mehdi Hassan	05:52	0.7	0.92	-8.935
3	Row2	Ek Tanete Jernc	Kishore Kumar	04:48	0.693	0.712	-12.256
4	Row3	Ami Ek Jajabar	Bhupen Hazarika	03:05	0.153	0.613	-9.27
5	Row4	Jina Rakhiya Te	Narendra Chanc	10:30	0.94	0.39	-9.421
6	Row5	Ham Hi Main T	Mehdi Hassan	15:26	0.73	0.315	-9.398
7	Row6	Navvuthaaru	S. P. Balasubrah	02:52	0.582	0.509	-3.247
8	Row7	Mohan Varni M	Usha Mangeshi	02:37	0.24	0.02	-8.567
9	Row8	Maguva	S. Janaki	03:13	0.715	0.81	-10.995

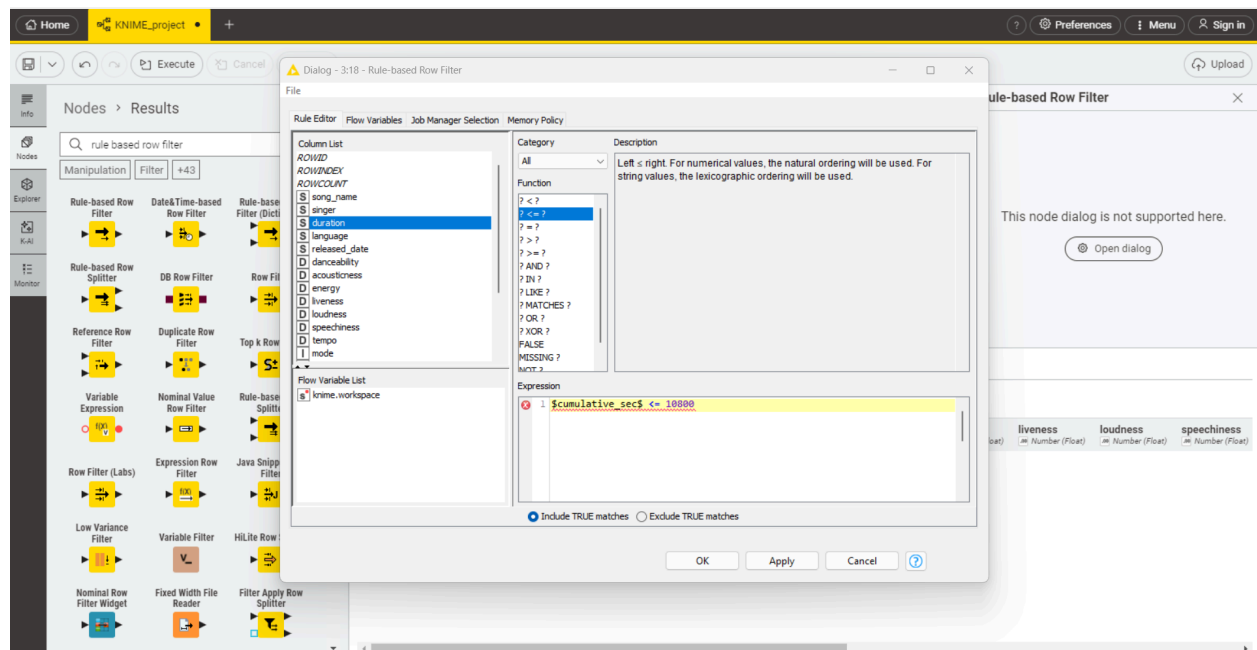
I connected the RowID node to the shuffle nodes so that shuffle can rearrange the order of rows without rewriting the ID or any other information. Furthermore, I created a pattern table that repeats the structure of **four high-danceability songs followed by one medium-danceability song**. This pattern maintains high energy while allowing short breaks for listeners. This resulted in concatenating the two separate playlists as we needed a final combined playlist.



To track overall runtime, I used the **Moving Aggregation** node with the aggregation set to **Sum (cumulative)** and a window configured as **All previous rows**. This calculated the running duration of the playlist by summing song lengths sequentially. During this step I realized that I must convert the time, a string component, into mm ss.



I did have trouble with the moving aggregation node, therefore I tried using a different method which involved using a string manipulation node which converted the time (string) into mm ss and then plugged the rule based row filter. Doing this step made me realize the assignment required the moving aggregation and without it no cumulative average is actually calculated



This limited the final playlist duration to approximately **3 hours** (10,800 seconds).

